

Soal 1

Enkripsikan MAGANG dengan $K = \begin{bmatrix} 7 & 6 \\ 2 & 5 \end{bmatrix}$. Tuliskan setiap langkah-langkahnya!

Jawaban:

1. Plaintext terdiri dari 6 suku kata. Karena bilangan prima terkecil dari 6 adalah 2, maka plaintext dikelompokkan ke dalam grup yang berisi 2 anggota.

MAGANG = MA - GA - NG

2. Tentukan representasi angka dari MAGANG dengan A = 0

M	12
A	0
G	6
A	0
N	13
G	6

3. Untuk masing-masing grup plaintext, lakukan perkalian matriks key dengan matriks grup kemudian moduluskan dengan 26 untuk mendapatkan angka yang akan diubah ke huruf yang disebut sebagai chipper text nya

PlainText					Mk * Mp	Mod 26	ChipperText
MA	7	6	.	12	84	6	G
	2	5		0	24	24	Y
GA	7	6	.	6	42	16	Q
	2	5		0	12	12	M
NG	7	6	.	13	127	23	X
	2	5		6	56	4	E

4. Gabungkan semua chipper text dari semua group

CHIPPER HASIL

GYQMXE

Soal 2

Enkripsikan nama lengkap anda menggunakan Affine Cipher dan kembalikan menjadi plaintexts, $a=9$ $b=[2 \text{ digit NPM akhir}]$

Jawab:

1. Lengkapi informasi soal:
Nama lengkap: NADYA HAPPY SIAHAAN
2 digit akhir NPM : 68
2. Untuk setiap karakter pada plain text, tentukan representasi angkanya (x) kemudian lakukan operasi $(ax+b) \bmod 26$ untuk mendapatkan y. Angka y akan dikembalikan ke bentuk huruf.

Plain Text	Angka rep. (x)	$ax+b$	$\bmod 26$ (y)	Chipper Text (y)
N	13	185	3	D
A	0	68	16	Q
D	3	95	17	R
Y	24	284	24	Y
A	0	68	16	Q
H	7	131	1	B
A	0	68	16	Q
P	15	203	21	V
P	15	203	21	V
Y	24	284	24	Y
S	18	230	22	W
I	8	140	10	K
A	0	68	16	Q
H	7	131	1	B
A	0	68	16	Q
A	0	68	16	Q
N	13	185	3	D

3. Untuk proses dekripsi, tentukan nilai a invers:

GCD(a,m) = GCD(9, 26)	
26 = 9*2 + 8	
9 = 8*1 + 1	
1 = 1*1 + 0	
q1 = 2	
q2 = 1	
$t_n = (t_{n-2} - (q_{n-1} \cdot t_{n-1})) \bmod m$	
t0 = 0	
t1 = 1	
t2 = (t0 - (q1 . t1)) mod 26 = 24	
t3 = (t1 - (q2 . t2)) mod 26 = 3	
Maka invers a = 3	

4. Lakukan dekripsi dengan melakukan operasi $D(y) = a^{-1} (y - b) \bmod 26$. Hasil $D(y)$ kemudian dikembalikan ke dalam bentuk alfabet hingga menghasilkan plain text yang sama dengan pesan sebenarnya.

mod 26 (y)	Chipper Text (y)	$a^{-1} * (y-b)$	mod 26	Plain Text	Cek Hasil
3	D	-195	13	N	Sama
16	Q	-156	0	A	Sama
17	R	-153	3	D	Sama
24	Y	-132	24	Y	Sama
16	Q	-156	0	A	Sama
1	B	-201	7	H	Sama
16	Q	-156	0	A	Sama
21	V	-141	15	P	Sama
21	V	-141	15	P	Sama
24	Y	-132	24	Y	Sama
22	W	-138	18	S	Sama
10	K	-174	8	I	Sama
16	Q	-156	0	A	Sama
1	B	-201	7	H	Sama
16	Q	-156	0	A	Sama
16	Q	-156	0	A	Sama
3	D	-195	13	N	Sama

Hasil pengerjaan di Spreadsheet:

https://docs.google.com/spreadsheets/d/1ccGXq9zZbc-0lwbDDoz6_PXoaBf2IAzi8FLW1ycaXvM/edit?usp=sharing

Soal 3

Buatlah program untuk enkripsi, dekripsi, dan mencari kunci Hill Cipher (bahasa pemrograman bebas)

Source code:

```
import math

MODULUS = 26

def text_to_numbers(text):
    text = text.upper().replace(" ", "")
    if not text.isalpha() or not text.isascii():
        raise ValueError("Teks hanya boleh berisi huruf A-Z.")
    return [ord(character) - ord("A") for character in text]

def numbers_to_text(numbers):
    return "".join(chr((number % MODULUS) + ord("A")) for number in numbers)

def input_key(order=None):
    if order is None:
        values = input("Masukkan semua elemen key (pisahkan spasi): ").split()
        element_count = len(values)
        order = math.isqrt(element_count)
        if order < 2 or order * order != element_count:
            raise ValueError("Jumlah elemen key harus kuadrat, minimal 4 angka.")
    else:
        values = []
        print(f"Masukkan key {order} x {order} (setiap baris dipisahkan spasi):")
        for row_number in range(order):
            row = input(f"Baris {row_number + 1}: ").split()
            if len(row) != order:
                raise ValueError(f"Setiap baris key harus berisi {order} angka.")
            values.extend(row)

    if len(values) != order * order:
        raise ValueError("Jumlah elemen key tidak sesuai dengan ordo matriks.")
    numbers = [int(value) % MODULUS for value in values]
    return [numbers[row * order:(row + 1) * order] for row in range(order)]

def determinant(matrix):
    size = len(matrix)
    if size == 1:
        return matrix[0][0] % MODULUS
    result = 0
    for column in range(size):
        minor = [
            [matrix[row][item] for item in range(size) if item != column]
            for row in range(1, size)
        ]
        sign = 1 if column % 2 == 0 else -1
        result += sign * matrix[0][column] * determinant(minor)
    return result % MODULUS

def validate_key(key):
    if math.gcd(determinant(key), MODULUS) != 1:
        raise ValueError("Key tidak valid: determinannya harus relatif prima dengan 26.")
```

```
def modular_inverse(number):
    for candidate in range(MODULUS):
        if number * candidate % MODULUS == 1:
            return candidate
    raise ValueError("Invers modular tidak ditemukan.")

def inverse_key(key):
    validate_key(key)
    size = len(key)
    augmented = [
        [value % MODULUS for value in key[row]]
        + [1 if row == column else 0 for column in range(size)]
        for row in range(size)
    ]

    for column in range(size):
        pivot = next(
            (row for row in range(column, size)
             if math.gcd(augmented[row][column], MODULUS) == 1),
            None,
        )
        if pivot is None:
            raise ValueError("Key tidak memiliki invers modulo 26.")
        augmented[column], augmented[pivot] = augmented[pivot], augmented[column]
        pivot_inverse = modular_inverse(augmented[column][column])
        augmented[column] = [
            value * pivot_inverse % MODULUS for value in augmented[column]
        ]
        for row in range(size):
            if row == column:
                continue
            factor = augmented[row][column]
            augmented[row] = [
                (augmented[row][item] - factor * augmented[column][item]) %
                MODULUS
                for item in range(size * 2)
            ]
    return [row[size:] for row in augmented]

def multiply_matrix_vector(matrix, vector):
    size = len(matrix)
    return [
        sum(matrix[row][column] * vector[column] for column in range(size))
        % MODULUS
        for row in range(size)
    ]

def process_text(text, key):
    block_size = len(key)
    numbers = text_to_numbers(text)
    numbers += [ord("X") - ord("A")] * ((-len(numbers)) % block_size)
    result = []
    for index in range(0, len(numbers), block_size):
        result.extend(multiply_matrix_vector(key, numbers[index:index
        block_size]))
    return numbers_to_text(result)

def encryption():
    plain_text = input("Masukkan plaintext: ")
    key = input_key()
    validate_key(key)
    print(f"CIPHERTEXT: {process_text(plain_text, key)}")
```

```
def decryption():
    cipher_text = input("Masukkan ciphertext: ")
    key = input_key()
    print(f"PLAINTEXT: {process_text(cipher_text, inverse_key(key))}")

def multiply_matrix(left, right):
    size = len(left)
    return [
        [
            sum(left[row][item] * right[item][column] for item in range(size))
            % MODULUS
            for column in range(size)
        ]
        for row in range(size)
    ]

def find_key():
    order = int(input("Masukkan ordo matriks key: "))
    if order < 2:
        raise ValueError("Ordo matriks minimal 2.")
    plain_text = input(
        f"Masukkan {order * order} huruf plaintext yang diketahui: "
    )
    cipher_text = input(
        f"Masukkan {order * order} huruf ciphertext pasangannya: "
    )
    plain_numbers = text_to_numbers(plain_text)
    cipher_numbers = text_to_numbers(cipher_text)
    required_length = order * order
    if len(plain_numbers) != required_length or len(cipher_numbers) !=
required_length:
        raise ValueError(f"Plaintext dan ciphertext harus tepat {required_length}
huruf.")

    # Setiap blok sepanjang ordo adalah satu vektor kolom.
    plain_matrix = [
        [plain_numbers[row + column * order] for column in range(order)]
        for row in range(order)
    ]
    cipher_matrix = [
        [cipher_numbers[row + column * order] for column in range(order)]
        for row in range(order)
    ]
    key = multiply_matrix(cipher_matrix, inverse_key(plain_matrix))
    validate_key(key)
    print("KEY:")
    print(f"{key[0][0]} {key[0][1]}")
    print(f"{key[1][0]} {key[1][1]}")

def menu():
    print("\n=== HILL CIPHER ===")
    print("1. Enkripsi")
    print("2. Dekripsi")
    print("3. Cari kunci")
    print("4. Keluar")
    return input("Masukkan pilihan: ").strip()

def main():
    actions = {"1": encryption, "2": decryption, "3": find_key}
    while True:
        pilihan = menu()
```

```
        if pilihan == "4":
            print("Program selesai.")
            return
        action = actions.get(pilihan)
        if action is None:
            print("Pilihan tidak valid.")
            continue
        try:
            action()
        except (ValueError, TypeError) as error:
            print(f"Error: {error}")

if __name__ == "__main__":
    main()
```

Output:

```
=== HILL CIPHER ===
1. Enkripsi
2. Dekripsi
3. Cari kunci
4. Keluar
Masukkan pilihan: 1
Masukkan plaintext: testing
Masukkan semua elemen key (pisahkan spasi): 1 2 2 7
CIPHERTEXT: BOENIDAR

=== HILL CIPHER ===
1. Enkripsi
2. Dekripsi
3. Cari kunci
4. Keluar
Masukkan pilihan: 2
Masukkan ciphertext: boenidar
Masukkan semua elemen key (pisahkan spasi): 1 2 2 7
PLAINTEXT: TESTINGX

=== HILL CIPHER ===
1. Enkripsi
2. Dekripsi
3. Cari kunci
4. Keluar
Masukkan pilihan: 3
Masukkan ordo matriks key: 2
Masukkan 4 huruf plaintext yang diketahui: test
Masukkan 4 huruf ciphertext pasangannya: boen
KEY:
1 2
2 7

=== HILL CIPHER ===
1. Enkripsi
2. Dekripsi
3. Cari kunci
4. Keluar
Masukkan pilihan: 4
Program selesai.
```